Ripasso: variabili e operatori

Lezione 1 — 4MAK, SST-INF-01c — Firmware PIC: ripasso pre-stage

21/09/2026

1 I tipi di dato: dove il microcontrollore tiene i numeri

Quando programmiamo, abbiamo bisogno di **salvare i dati** che vogliamo elaborare in **variabili**: a livello hardware, sono **registri di memoria** dentro al microcontrollore. Le variabili possono avere dimensioni diverse in base al numero che devono contenere.

Definizione

char: occupa **1 byte** (8 bit), contiene numeri interi da 0 a 255.
int: occupa (su microcontrollori a 8 bit) minimo **16 bit**, quindi 65.536 combinazioni. Metà delle combinazioni rappresenta i numeri positivi (da 0 a 32.767), metà i negativi (da -1 a -32.768).
float: occupa **32 bit** (circa 4 miliardi di combinazioni), va da circa -2 a $+2$ miliardi, e permette numeri **con la virgola**.

Attenzione

Char e int contengono **solo numeri interi**: se si assegna un valore con la virgola (es. `int a = 12.99;`), la parte decimale viene **troncata**, **non arrotondata** – `a` conterrà 12, non 13.

Attenzione

Su un microcontrollore a 8 bit come il PIC, manipolare i `float` è **molto pesante** in termini di calcolo: la macchina deve eseguire molte più operazioni interne. Vanno **evitati** quando possibile – char e int sono nativi e molto più leggeri da gestire.

1.1 Le regole per i nomi delle variabili

Definizione

1. Il nome è una sequenza **alfanumerica** (**lettere e numeri**).
2. È **case sensitive**: `pippo` e `Pippo` sono due variabili diverse.
3. Non può **iniziare con un numero**: `Pippo3` è valido, `3Pippo` no – dà **errore di compilazione**.
4. Non può contenere **caratteri speciali**, a eccezione dell'**underscore** (_): ad esempio un punto di domanda o un **trattino** "meno" in mezzo al nome rendono la variabile non valida.
5. Non può essere una **parola riservata** del linguaggio (es. `int`, `return`, `for`).

Nota

Si può iniziare un nome con l'**underscore**, ma nel linguaggio C le variabili che iniziano così hanno un significato tecnico particolare (direttive di sistema): meglio evitarlo, per non creare confusione con codice scritto da altri.

2 Gli operatori: la macchina sa fare solo calcoli

Che sia un computer, un Arduino o un **microcontrollore PIC**, la macchina sa fare una cosa sola: **calcoli**. Anche un rendering 3D complesso in un videogioco è, alla base, il risultato di **milioni di calcoli**.

Definizione

Un **operatore** prende due (o più) elementi e li **combina** secondo una relazione: più, meno, per, diviso, and, or, not, maggiore, minore...

2.1 Operatore di assegnazione

Definizione

Il simbolo = **copia** il valore a destra dentro alla **variabile a sinistra** – non è l'uguaglianza matematica.

Attenzione

L'uso scorretto dell'uguale singolo al posto del confronto (==) è uno degli **errori più insidiosi** in programmazione: spesso non genera un **errore di compilazione**, ma fa eseguire il programma in modo **silenziosamente sbagliato**, sovrascrivendo un valore che si voleva solo confrontare – un po' come salvare un file nuovo sopra uno vecchio con lo stesso nome: il contenuto precedente va perso senza nessun avviso.

2.2 Operatori aritmetici e modulo

Definizione

Somma (+), sottrazione (-), moltiplicazione ($*$), divisione (/). L'operatore **modulo** (%) dà il **resto** di una divisione **tra interi** – vale solo tra tipi interi (char/int), **mai con i float**.

Esempio

$5 \div 2 = 2$ con resto 1, quindi $5 \bmod 2 = 1$.
$40 \bmod 9$? $9 \times 4 = 36$, e $40 - 36 = 4$: quindi $40 \bmod 9 = 4$.

2.3 Operatori di confronto

Definizione

A differenza degli aritmetici, gli **operatori di confronto** (>, >=, <, <=, ==, !=) restituiscono solo vero o **falso**. In informatica, il valore **0** è sempre **falso**; **qualunque altro numero** (positivo o negativo) è **vero**.

Attenzione

Il doppio uguale (==) è il **vero simbolo di confronto**/uguaglianza, diverso dal singolo = che **copia un dato**. Confonderli è un errore comune e insidioso.

2.4 Operatori logici: per valore e bitwise

Definizione

Gli operatori logici AND (&&), OR (||), NOT (!) **"per valore"** guardano il **valore numerico complessivo** di una variabile (0 = falso, tutto il resto = vero).
Gli operatori **bitwise** (AND &, OR |, NOT ~) lavorano invece **bit per bit** sulla **rappresentazione binaria** dei numeri.

Esempio

$A = 5$ (binario `0101`), $B = 10$ (binario `1010`). L'**AND bitwise** (`A & B`) confronta bit a bit come una moltiplicazione: $1{\times}0 = 0$, $0{\times}1 = 0$, $1{\times}0 = 0$, $0{\times}1 = 0$ – risultato `0000`, cioè falso.
L'**AND "per valore"** (`A && B`) invece guarda solo se A e B sono **diversi da zero**: entrambi lo sono, quindi il risultato è vero. **Stessi numeri, risultato diverso**: ecco perché è fondamentale distinguere le due categorie.
L'**OR bitwise** (`A | B`) funziona allo stesso modo ma con la somma booleana bit a bit (dove $1 + 1 = 1$, non 2): $0{+}1 = 1$, $1{+}0 = 1$, $0{+}1 = 1$, $1{+}0 = 1$ – risultato `1111`.
Il **NOT bitwise** (`~A`) **inverte tutti i bit**: se $A = 5$ (`0101`), `~A` dà `1010`, cioè 10 – un valore che qui non ha un significato numerico diretto, perché quando si lavora sui bit non si ragiona più in cifre decimali ma sui singoli bit da accendere, spegnere o invertire.

Nota

Chi lavora su microcontrollori usa spesso gli **operatori bitwise** per accendere, spegnere o invertire **singoli bit di un registro** (es. per pilotare pin/LED), non per fare calcoli numerici in senso classico. Chi scrive software di alto livello raramente ne ha bisogno.

Attenzione

L'operatore **NOT** (sia logico che bitwise) si applica solo all'**operando immediatamente a destra**: scrivere `A NOT B` non ha significato. Per combinare correttamente gli operatori servono le **parentesi**, ad esempio `A && (!B)`.

2.5 Lo shift

Definizione

Lo **shift** (« a sinistra, » a destra) **trasla** tutti i bit di un numero di un certo numero di posizioni. I **bit che escono** da un lato si perdono; dall'altro lato entrano bit nuovi (solitamente 0).

Esempio

Il numero $A = 6$ su 8 bit si rappresenta come `00000110`. Shiftando `A 1` (a sinistra di 1 posizione) si ottiene `00001100`, cioè 12: il valore è **raddoppiato**. Shiftando invece `A 1` (a destra di 1) si perde l'ultimo bit e si ottiene `00000011`, cioè 3: il valore è **dimezzato**.
In generale, shiftare a sinistra di n posizioni **moltiplica per** 2^n; shiftare a destra di n posizioni **divide per** 2^n – vale solo per **potenze di 2** (2, 4, 8, 16, 32...), non per un moltiplicatore qualunque.

Attenzione

Molti microcontrollori economici non hanno un'**unità hardware dedicata** per moltiplicazione/divisione: eseguirle richiede **molti cicli macchina**. Lo shift, invece, è quasi immediato: quando il fattore è una **potenza di 2**, conviene sempre usare lo shift al posto della moltiplicazione/divisione esplicita, guadagnando molta velocità.

3 Cosa ricordare di questa lezione

- **char** (1 byte, 0-255), **int** (min. 16 bit, con segno), **float** (4 byte, con virgola) – **evitare i float** sui microcontrollori a 8 bit quando possibile.
- L'operatore di **assegnazione** (=) **copia da destra a sinistra**; il **confronto** (==) verifica l'uguaglianza – non vanno confusi.
- Il **modulo** (%) dà il **resto** di una divisione tra interi.
- Operatori logici **"per valore"** vs **bitwise**: stessi numeri, risultati diversi – i bitwise lavorano **bit per bit**.
- Lo **shift** è un modo veloce per **moltiplicare/dividere per potenze di 2**.